Supplementary Materials:
LLM-Assisted Extraction of QSP Calibration Targets

Joel Eliason

Contents

1 S1: Complete SubmodelTarget Example

This section presents a complete calibration target for activated pancreatic stellate cell (aPSC) proliferation, demonstrating all schema components.

1.1 Overview

This target constrains the proliferation rate parameter `k_apsc_prolif` using data from Schneider et al. (2001), who measured PSC proliferation in response to PDGF stimulation via BrdU incorporation assay.

1.2 Complete YAML Specification

```yaml
# Schema:  SubmodelTarget
# Target:  Activated PSC proliferation rate

target_idpsc_proliferation_PDAC_deriv001

# ============================================================================
# INPUTS: Extracted values with full provenance
# ============================================================================
inputs:
  -namepdgf_fold_mean
    value4.37
    unitsdimensionless
    input_typedirect_measurement
    roletarget
    source_refSchneider2001_PSC_PDGF
    source_locationAbstract
    value_snippet"Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                0.39-fold of control)."

  -name: pdgf_fold_sd
    value: 0.49
    units: dimensionless
    input_type: direct_measurement
    role: auxiliary
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                0.39-fold of control)."

# ============================================================================
# CALIBRATION: Everything needed for inference
# ============================================================================
calibration:
  parameters:
    -name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0 # log(1.0) -expect ~1/day for 4x fold-change in 1 day
        sigma: 1.0
        rationale: |
          Wide log-normal prior centered at 1/day. A 4-fold increase in
          1 day implies k = ln(4) = 1.4/day, so prior median of 1/day
          is reasonable.

  state_variables:
    -name: aPSC_rel
      units: dimensionless
      initial_condition:
        value: 1.0
```

```
      rationale: |
        PSC proliferation data are reported as fold of control. We
        normalize the initial aPSC population to 1.0 (100% control)
        to match this convention.

model:
  type: exponential_growth
  rate_constant: k_apsc_prolif
  data_rationale: |
    BrdU incorporation assay measures proliferation as fold-change
    over ~24 hours under constant PDGF stimulus. With a single high
    PDGF dose and short duration, exponential growth (dN/dt = k*N)
    fits the data structure.
  submodel_rationale: |
    The full QSPIO_PDAC model has aPSC proliferation via:
    k_apsc_prolif * aPSC * (PDGF / (PDGF_50_prolif + PDGF)).
    At 50 ng/mL PDGF >>PDGF_50_prolif, the saturation term ~ 1,
    reducing to exponential growth with rate k_apsc_prolif.

independent_variable:
  name: time
  units: day
  span: [0.0, 1.0]
  rationale: |
    Proliferation assay duration assumed to be ~24 hours based on
    typical cytokine incubation protocols for rat PSCs.

measurements:
  -name: proliferation_fold_change
    observable:
      type: identity
      state_variables: [aPSC_rel]
      rationale: |
        The state variable aPSC_rel is the fold-change in aPSC number
        relative to baseline (control = 1.0), which directly matches
        the reported proliferation fold-change.
    units: dimensionless
    uses_inputs:
      -pdgf_fold_mean
      -pdgf_fold_sd
    evaluation_points: [1.0]
    sample_size: 3
    sample_size_rationale: |
      Sample size not explicitly reported; assumed n=3 independent
      experiments, typical for in vitro PSC BrdU proliferation assays.
    distribution_code: |
      def derive_distribution(inputs, ureg):
          import numpy as np

          rng = np.random.default_rng(42)

          mean = inputs['pdgf_fold_mean'].magnitude
          sd = inputs['pdgf_fold_sd'].magnitude
          n = 10000

          # Use lognormal for positive fold-changes
          mu_log = np.log(mean**2 / np.sqrt(mean**2 + sd**2))
          sigma_log = np.sqrt(np.log(1 + sd**2 / mean**2))

          samples = rng.lognormal(mu_log, sigma_log, n)

          median_val = np.median(samples)
          ci_lower = np.percentile(samples, 2.5)
          ci_upper = np.percentile(samples, 97.5)

          return {
              'median': [median_val],
              'ci95': [[ci_lower, ci_upper]],
```

```yaml
                'units': 'dimensionless'
            }
      likelihood:
        distribution: lognormal
        rationale: |
          Fold-changes are positive and multiplicative, making
          lognormal appropriate.

  identifiability_notes: |
    With a single timepoint (~24h) and normalized initial condition
    (aPSC_rel(0)=1), only k_apsc_prolif is estimated. If substantial
    PSC death occurred over the assay interval, the inferred rate
    would represent net growth rather than pure proliferation.

# ==============================================================================
# EXPERIMENTAL CONTEXT
# ==============================================================================
experimental_context:
  species: rat
  system: in_vitro_primary_cells
  indication: PDAC
  cell_types:
    -name: pancreatic stellate cell
      phenotype: activated (alpha-SMA positive)
      isolation_method: Isolated from normal Wistar rat pancreas,
                        activated by culture on plastic
  culture_conditions:
    culture_type: 2d_monolayer

# ==============================================================================
# STUDY INTERPRETATION
# ==============================================================================
study_interpretation: |
  Schneider et al. (2001) measured proliferation of cultured rat
  pancreatic stellate cells (PSCs) in response to growth factors using
  BrdU incorporation. Addition of 50 ng/mL PDGF increased PSC
  proliferation to 4.37 +/- 0.49-fold of control, indicating that PDGF
  acts as a strong mitogen for activated PSCs. We interpret the
  50 ng/mL PDGF condition as approximating a near-maximal PDGF stimulus
  and use the reported fold-change over ~24 hours as a quantitative
  constraint on the maximal PDGF-driven proliferation rate parameter
  k_apsc_prolif.

key_assumptions:
  -|
    50 ng/mL PDGF corresponds to a near-saturating concentration for
    PSC proliferation, so PDGF/(PDGF_50_prolif + PDGF) ~ 1.
  -|
    The proliferation assay duration is approximately 24 hours,
    consistent with typical cytokine incubation protocols.
  -|
    BrdU incorporation fold-change is proportional to fold-change in
    PSC cell number (negligible death over 24 hours).
  -|
    Rat activated PSC proliferation kinetics in vitro provide an upper
    bound for human tumor aPSC proliferation in vivo.

key_study_limitations:
  -|
    Proliferation was measured via BrdU incorporation rather than
    direct cell counts.
  -|
    Single PDGF concentration and single assay interval constrain
    only the maximal rate.
  -|
    Data from rat PSCs in vitro, not human PDAC-associated stellate cells.
  -|
    Background serum proliferative signals not explicitly modeled.
```

```yaml

# ==============================================================================
# DATA SOURCES
# ==============================================================================
primary_data_source:
  doi: "10.1152/ajpcell.2001.281.2.C532"
  title: "Identification of mediators stimulating proliferation and
         matrix synthesis of rat pancreatic stellate cells"
  authors: [Schneider, Schmid, Liptay, Schmid, Pohl]
  year: 2001
  source_tag: Schneider2001_PSC_PDGF

secondary_data_sources:
  -doi: "10.1136/gut.44.4.534"
    title: "Pancreatic stellate cells are activated by proinflammatory
           cytokinesimplications for pancreatic fibrogenesis"
    authors[Apte, Haber, Darby, Rodgers, McCaughan, Korsten, Pirola, Wilson]
    year1999
    source_tagApte1999_PSC_Cytokines
    contribution|
      Provides context on PSC activation and cytokine response protocols.
```

1.3 Schema Structure Summary

The SubmodelTarget schema has the following top-level structure:

- **target_id**: Unique identifier for the calibration target
- **inputs**: List of values extracted from literature with provenance
- **calibration**: Everything needed for inference code generation
  - `parameters`: Parameters to estimate with priors
  - `state_variables`: ODE state variables with initial conditions
  - `model`: Model type specification (built-in or custom)
  - `independent_variable`: Time or other independent variable
  - `measurements`: Observables with likelihoods
  - `identifiability_notes`: Discussion of parameter identifiability
- **experimental_context**: Species, system, cell types, culture conditions
- **study_interpretation**: Scientific interpretation of the data
- **key_assumptions**: Assumptions made in using this data
- **key_study_limitations**: Known limitations of the source study
- **primary_data_source**: Primary literature reference with DOI
- **secondary_data_sources**: Additional references

2 S2: Supported Model Types

The schema supports the following built-in model types:

Table 1: Built-in model types with their ODE formulations.

Model Type	ODE	Parameters
`exponential_growth`	$dy/dt = k \cdot y$	`rate_constant`
`first_order_decay`	$dy/dt = -k \cdot y$	`rate_constant`
`logistic`	$dy/dt = k \cdot y \cdot (1 - y/K)$	`rate_constant`, `carrying_capacity`
`saturation`	$dy/dt = k \cdot (1 - y)$	`rate_constant`
`two_state`	$dA/dt = -k \cdot A$, $dB/dt = k \cdot A$	`forward_rate`
`michaelis_menten`	$dy/dt = -V_{max} \cdot y/(K_m + y)$	`vmax`, `km`
`direct_conversion`	(analytical formula)	`formula`
`direct_fit`	(curve fitting)	`curve`
`custom`	(user-provided)	`code`, `code_julia`

3 S3: Validation Checks

The validation framework includes the following automated checks:

1. **Input reference validation**: All `uses_inputs` references match defined input names
2. **Source reference validation**: All `source_ref` values match defined source tags
3. **Parameter reference validation**: Model parameter roles reference defined parameters
4. **State variable reference validation**: Observable `state_variables` match defined state variables
5. **DOI resolution**: All DOIs resolve via CrossRef API
6. **Value-in-snippet validation**: Extracted values appear verbatim in quoted snippets
7. **Unit validation**: All unit strings parse with Pint
8. **Code syntax validation**: Custom code passes Python AST parsing
9. **Span ordering validation**: Time spans have start < end
10. **ODE model requirements**: ODE models have required state variables and spans

4 S4: Generated Julia Code Structure

The translator generates Julia scripts with the following structure:

```julia
using DifferentialEquations
using Turing
using Distributions

# Observed data (from inputs layer)
const obs_proliferation_median = 4.37
const obs_proliferation_sigma = 0.25 # derived from CI95

# ODE function
function ode_proliferation!(du, u, p, t)
    k = p[1]
    du[1] = k * u[1] # exponential growth
```

```
end

# Simulate function
function simulate_proliferation(k; t_span=(0.0, 1.0), u0=[1.0])
    prob = ODEProblem(ode_proliferation!, u0, t_span, [k])
    sol = solve(prob, Tsit5())
    return sol[end][1] # final value
end

# Turing model
@model function proliferation_model()
    # Prior
    k_apsc_prolif ~ LogNormal(0.0, 1.0)

    # Simulate
    pred = simulate_proliferation(k_apsc_prolif)

    # Likelihood
    obs_proliferation_median ~ LogNormal(log(pred), obs_proliferation_sigma)
end

# Run inference
model = proliferation_model()
chain = sample(model, NUTS(), MCMCThreads(), 1000, 4)
```

For joint inference over multiple targets, the translator identifies shared parameters by name and generates a single model with one prior per unique parameter.